I. Overview

This code is a comprehensive Python code for detecting, labeling and tracking particles in videos of active and passive granular systems. It includes:

- Color-specific particle detection
- Ellipse fitting to particle contours
- Particle tracking across frames
- Coordination number and cluster network analysis
- Per-frame summary statistics
- Output generation for visualization and further analysis

II. Dependencies

It is better to ensure the following libraries are installed before running the code (you can use pip install <package> if missing):

- numpy
- opencv-python
- scipy
- matplotlib
- PIL (Python Image Library
- networkx
- python-louvain
- scikit-image

The input requirements are the following

- A video file (e.g., .MOV, .MP4) capturing the granular particles under analysis
- Predefined color thresholds for detecting particles in various color spaces (HSV, RGB)
- A bounding box (region of interest or ROI) that isolates the area of interest
- Optional wall coordinates to visualize boundaries

The output files that the code produce are saved in a user-defined folder (e.g., Results169/) and include:

1. LabeledFrames/: Frames annotated with ellipses and particle IDs

2. BinaryImages/: Thresholded binary masks for different particle types

3. OutputFiles/: Per-frame particle data (coordinates, sizes, clustering metrics)

4. IDData/: Reserved for future ID-based statistics

5. ContactNetworks/: Images of particle contact graphs

6. SystemSummary.txt: Summary of system-wide stats for each frame

III. Main Workflow

The code performs the following steps:

1. Open and process each frame of the input video
2. Crop the frame to the region of interest (ROI)
3. Apply color thresholding in RGB and HSV to identify particles of different types
4. Detect particle contours and fit ellipses to them
5. Match particles across frames to maintain consistent IDs
6. Calculate physical and network-based features:

- Coordination number
- Cluster assignment
- Cluster density
- Clustering coefficient
- Betweenness centrality
- Louvain modularity
- Global efficiency

7. Save binary masks, labeled images, network visualizations and per-frame data.

IV. Function Descriptions

A. get_contours(binary, min_dist=25, min_area=2500)

It cleans the binary image and applies morphological operations. It detects particles via distance transform + watershed, filters small noise.

B. get_particle_list(binary, min_dist=25)

It calls get_contours and fits ellipses to each contour. It returns particle descriptors and outlines.

C. label_image(img, frame_no, ...)

It draws particle IDs, ellipses, contours and optional wall rectangles on the image.

D. match_particles(prev_particles, curr_particles, prev_ids, next_id, max_dist=200)
It assigns consistent particle IDs across frames using centroid matching (Hungarian algorithm).
E. analyze_movies(...) → IMPORTANT NOTE: It is adjustable to include more and more types of particles. Check the commented out parts to see what would be needed to reduce or increase the number of particle-types being detected.
It is the main function that loops over the video:
- Detects particles across multiple color ranges
- Tracks IDs
- Computes contact networks
- Saves visualizations and numerical outputs

V. Example Usage → Notice that the analyze_movie function was updated to detect four kinds of particles

```
# IMPORTANT NOTE: Depending on the type of color, change in_range accordingly

walls_fixed = [(0,0), (1145,0), (1145,1030), (0,1030)] # The origin is on the top left
white_range = [(155, 155, 155), (255, 255, 255)]   # Transparent/white
#yellow_range = [[18, 70,70], [70, 255, 255]] # Yellow
yellow_range = [[18, 70, 70], [70, 255, 255]]
green_range = [[65, 60, 40], [85, 255, 255]]
red_range1 = [[0, 80, 80], [8, 255, 255]]
red_range2 = [[142, 100, 100], [180, 255, 255]]

#analyze_movies("DSC_0147.MOV", "Results147/", 245, 35, 1400, 1050, in_range1=white_range, in_range2=green_range,walls=walls_fixed )
analyze_movies("DSC_0169.MOV", "Results169/", 253, 0, 1386, 1030, in_range1=white_range,in_range2=yellow_range, in_range3=green_range, in_range4a=red_range1, in_range4b = red_range2, walls=walls_fixed )
```

# IMPORTANT NOTE: Depending on the type of color, change in_range accordingly
walls_fixed = [(0,0), (1145,0), (1145,1030), (0,1030)] # The origin is on the top left
white_range = [(155, 155, 155), (255, 255, 255)]   # Transparent/white
#yellow_range = [[18, 70,70], [70, 255, 255]] # Yellow
yellow_range = [[18, 70, 70], [70, 255, 255]]
green_range = [[65, 60, 40], [85, 255, 255]]
red_range1 = [[0, 80, 80], [8, 255, 255]]
red_range2 = [[142, 100, 100], [180, 255, 255]]
#analyze_movies("DSC_0147.MOV", "Results147/", 245, 35, 1400, 1050, in_range1=white_range, in_range2=green_range,walls=walls_fixed)
analyze_movies("DSC_0169.MOV", "Results169/", 253, 0, 1386, 1030, in_range1=white_range,in_range2=yellow_range, in_range3=green_range, in_range4a=red_range1, in_range4b = red_range2, walls=walls_fixed)

VI. Customization Tips and Important Notes

A. Adjusting min_area in get_contours()

- One can tune these ranges using a color picker or using OpenCV sliders to isolate the particle type more effectively.
- Always change walls_fixed, xx, yy, ww, and hh to match the physical setup of your experiment.

B. Extending to More Colors

- Add additional in_rangeX and binaryX definitions for more particle types. Don't forget to modify the analyze_movies() function accordingly.
- If one sees more motion than anticipated, please adjust the particle matching tolerance. The max_dist parameter in match_particles can be adjusted if particles move significantly between frames.

C. Important Notes

- Color-based segmentation works best under stable lighting conditions
- The code assumes that particles do not enter or exit the frame between frames rapidly.
- I highly recommend running the code for the first few frames and checking the labeled frames to ensure it is doing a decent job, otherwise, it will be a waste of running time.